

Global Cybersecurity Severity Prediction

Competition Report

BASTIANELLO Gabriel
EVELBAUER Beatriz
SLIMEN Yasmine

March 7, 2026

Project Links

- Github repository: <https://github.com/yslimen7/cybersecurity-severity-prediction>
- Codabench competition: <https://www.codabench.org/competitions/14465/#/phases-tab>

1 Introduction

The **Global Cybersecurity Severity Prediction** competition is a machine learning challenge focused on predicting the severity of cybersecurity incidents from structured tabular features. The task is formulated as a **multi-class classification problem**, where each event must be assigned to one of four severity levels:

- Low
- Medium
- High
- Critical

The main goal of the competition is to build a model capable of learning the relationship between incident characteristics and the final severity category. From a practical perspective, such a system could help organizations prioritize incident response, estimate operational impact, and support risk management decisions.

This report presents the competition objective, the dataset structure, the way the labels were obtained, the methodological choices made in the starting notebook, and the rationale behind the baseline modeling pipeline.

2 Competition Description

The challenge consists of predicting the severity level of a cybersecurity event from a set of descriptive variables. These variables capture different aspects of an incident, including the attack context, the affected sector, the likely vulnerability, and operational impact indicators.

The problem is not a regression task, but a classification task with four ordered severity categories. Even though the labels may suggest an ordinal structure, the notebook treats the problem as a standard multi-class classification problem, which is a reasonable and simple baseline design.

The competition is implemented on **Codabench**, where participants submit code that is executed by an ingestion program. This program trains the model on the training data and generates predictions for the public and private test sets.

3 Dataset Description

3.1 Available Files

The repository is organized around the following files:

- Training features: `dev_phase/input_data/train/train_features.csv`
- Training labels: `dev_phase/input_data/train/train_labels.csv`
- Public test features: `dev_phase/input_data/test/test_features.csv`
- Private test features: `dev_phase/input_data/private_test/private_test_features.csv`

The ground-truth labels used for evaluation are stored separately in:

- Public labels: `dev_phase/reference_data/test_labels.csv`
- Private labels: `dev_phase/reference_data/private_test_labels.csv`

3.2 Feature Space

The dataset includes features such as:

- Country
- Year
- Attack Type
- Target Industry
- Number of Affected Users
- Attack Source
- Security Vulnerability Type
- Defense Mechanism Used
- Incident Resolution Time (in Hours)

These variables mix categorical and numerical information.

3.3 Origin of the Data

The dataset used in this competition was generated for educational purposes to simulate cybersecurity incidents occurring across different countries and industries.

It contains synthetic records describing attack characteristics, impacted sectors, and operational consequences of incidents. The dataset was prepared and structured using the script:

```
tools/setup_data.py
```

This script generates the training, public test, and private test splits used in the Codabench challenge.

4 Exploratory Data Analysis

An exploratory data analysis was conducted in the `starting_kit.ipynb` notebook to better understand the structure and characteristics of the dataset before training a machine learning model.

The training dataset contains **2386 samples** and **9 input features** describing simulated cybersecurity incidents. Each observation corresponds to a single incident and is associated with a severity label.

The exploratory analysis included:

- inspecting the dataset dimensions
- checking feature types (categorical vs numerical)
- verifying the presence of missing values
- analyzing the distribution of the target variable

The dataset contains both categorical variables (such as **Country**, **Attack Type**, **Target Industry**, **Attack Source**, **Security Vulnerability Type**, and **Defense Mechanism Used**) and numerical variables (including **Year**, **Number of Affected Users**, and **Incident Resolution Time (in Hours)**).

The analysis shows that the dataset contains **no missing values**, which simplifies the preprocessing stage.

In addition, the distribution of the severity labels is **almost perfectly balanced** across the four classes (**Low**, **Medium**, **High**, and **Critical**). This balanced distribution reduces the risk of bias toward a dominant class and supports the use of the Macro F1-score as the evaluation metric.

To further understand the relationship between the input variables and the severity label, we also analyzed the distributions of two key impact-related variables: the number of affected users and the incident resolution time. The boxplots presented in the notebook show that incidents with higher severity levels tend to affect more users and require longer resolution times.

These observations confirm that meaningful relationships exist between the features and the severity level, making the dataset well suited for a multi-class classification task.

5 How the Data and Labels Were Obtained

An important aspect of this project is that the original data did **not** contain a ready-to-use target variable called *Severity Level*. Instead, the label was created during the data preparation step using the script:

```
tools/setup_data.py
```

Based on the project notebook, the severity label was derived from impact-related indicators, especially:

- the **number of affected users**
- the **incident resolution time**

These two variables were used to construct a severity category with four classes: **Low**, **Medium**, **High**, and **Critical**. Conceptually, the transformation reflects the intuition that incidents affecting more users and requiring longer recovery times should be considered more severe.

This preprocessing step is important for two reasons. First, it transforms the raw incident data into a supervised learning problem. Second, it defines the target variable used by the model and therefore shapes the entire competition.

6 Evaluation Metric

The competition uses the **Macro F1-score** as the main evaluation metric, as described in the `starting_kit.ipynb` notebook.

The F1-score combines precision and recall into a single metric:

$$F1 = 2 \times \frac{precision \times recall}{precision + recall}$$

The Macro F1-score computes the F1-score independently for each class and then averages them.

This metric is particularly suitable for multi-class classification problems where class imbalance may occur, since it gives equal importance to all classes.

7 Conclusion

Overall, this competition provides a clear framework to experiment with machine learning methods for cybersecurity incident analysis while illustrating the full workflow of a benchmark challenge, from dataset preparation to model evaluation.